PROJECT SYNOPSIS
On

TEXT-BASED ADVENTURE GAME

RU-100-01-00021

Programming Fundamentals and Logic Building Using Python

SESSION: 2026–27

Submitted By:

Shoaib Akhtar

RU ID: RU-26-00647

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with IBM

Under the Guidance of:

Mr. Priyanshu

(IBM SME and EXPERT)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

Contents

1. Introduction and Problem Statement

2. Objectives

3. Concepts and Requirements Mapping

4. Program Design

5. Implementation

6. Testing

7. Sample Runs

8. Challenges and Solutions

9. Future Enhancements

10. Conclusion

11. References

1. Introduction and Problem Statement

Text-based adventure games were among the earliest computer games. The player reads descriptions and makes decisions by typing choices, and the program responds to each decision. Building one is a good first project because it needs only the core ideas of programming: taking input, making decisions, repeating actions and organising code into functions.

Problem statement. Create a text-based adventure game in which the player explores a forest, a cave and a river by choosing options from a menu, faces a random monster attack, collects score and ends the game by quitting or losing all health.

The game rules given in the assignment are:

- The player starts in the Forest with Health 100 and Score 0.
- Menu choices: 1 Cave, 2 River, 3 Quit.
- Cave: a monster attacks with random damage from 10 to 40 (fixed to 30 for testing). If the player survives, a key gives +20 score.
- River: a treasure gives +50 score.
- The game ends when the player quits or Health reaches 0.

2. Objectives

- Understand the problem and define it clearly.
- Design the solution with functions, conditions and a loop.
- Use the random module for monster damage and track Health and Score.
- Validate every menu choice so wrong input never breaks the game.
- Test the program with the four given test cases and additional edge cases.
- Document the work and publish the code and report on GitHub.

3. Concepts and Requirements Mapping

3.1 Programming concepts used

Concept	Where it is used	Purpose
Conditions	if / elif / else in play(), cave() and forest()	Choose the action for each menu option; check if the player survived
Loops	while health > 0 in play(); while True in forest()	Keep the game running; repeat the menu until the input is valid
Functions	forest(), cave(), river(), get_damage(), show_status(), play(), main()	Separate each place and task so the code is easy to read and test
Random	random.randint(10, 40) in get_damage()	Produce unpredictable monster damage

3.2 Bare minimum requirements checklist

Requirement	Implementation	Status
Python 3 solution in one .py file, run from the console	adventure_game.py	Done
Functions for each place; conditions for choices	forest(), cave(), river(); if / elif / else	Done
Loop until the player quits or Health reaches 0	while health > 0 in play()	Done
random module for damage; track Health and Score	get_damage(); score and health variables	Done
Validate every menu choice; show final score at the end	Validation loop in forest(); final score on quit and on Game Over	Done

4. Program Design

4.1 Algorithm

1. Start. Set Health = 100 and Score = 0.
2. While Health is greater than 0, show the Forest menu and read the choice.
3. If the choice is not 1, 2 or 3, print an error message and ask again. Health and Score do not change.
4. If the choice is 1 (Cave): generate the damage, reduce Health (never below 0) and print the attack. If Health is still above 0, add 20 to Score and print that a key was found.
5. If the choice is 2 (River): add 50 to Score and print the treasure message with the current Score and Health.
6. If the choice is 3 (Quit): print the final Score and Health and stop.
7. If Health reaches 0, the loop ends: print Game Over with the final Score and stop.

4.2 Flowchart

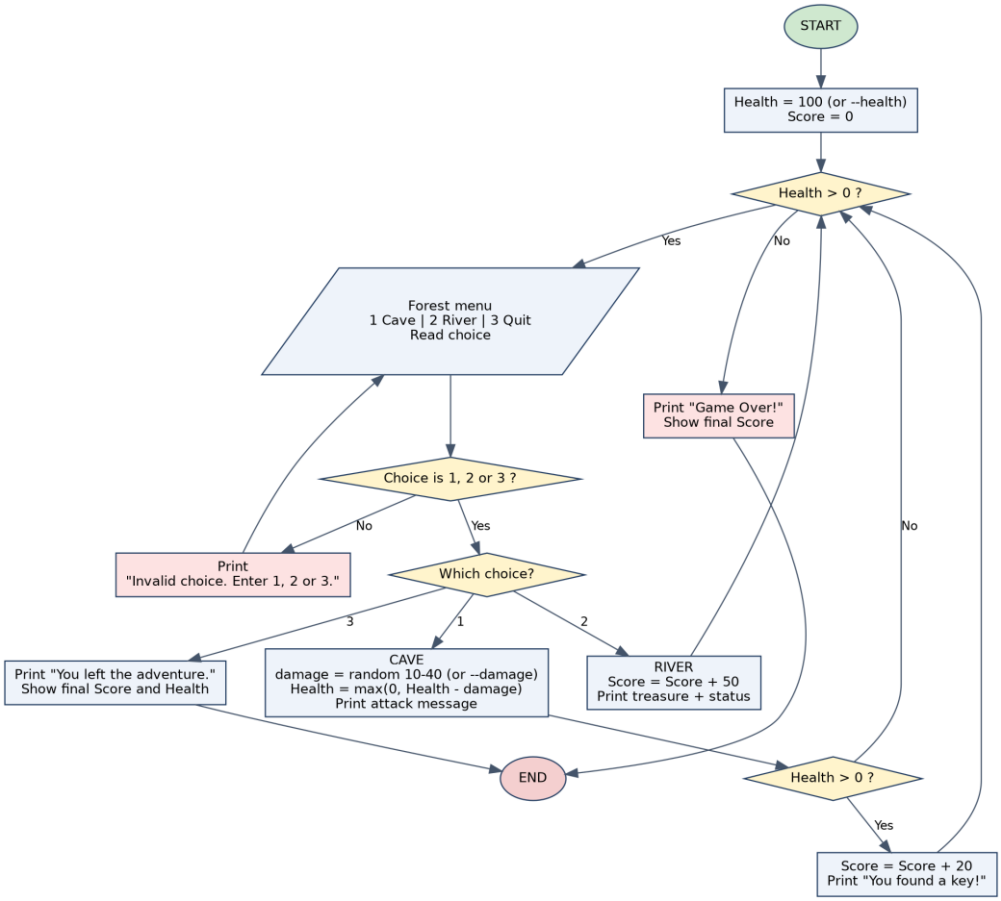


Figure 1: Flowchart of the Text-Based Adventure Game

4.3 Functions

Function	Description
`forest()`	Prints the menu and keeps asking until the player enters 1, 2 or 3. Returns the choice. Treats end of input (Ctrl+D) as Quit.
`cave(score, health, fixed_damage)`	Gets the monster damage, reduces health (minimum 0), prints the attack and gives +20 score if the player survives. Returns the new score and health.
`river(score, health)`	Adds +50 score, prints the treasure message and status. Returns the new score and health.
`get_damage(fixed_damage)`	Returns random.randint(10, 40), or the fixed value when testing.
`show_status(score, health)`	Prints the line: Score: X | Health: Y.
`play(health, fixed_damage)`	Main game loop. Runs until the player quits or Health reaches 0 and returns the final score and health.
`main()`	Reads the optional --health and --damage command-line options and starts the game.

4.4 Constants and variables

Name	Value	Meaning
`DEFAULT_HEALTH`	100	Starting health of the player
`MIN_DAMAGE` / `MAX_DAMAGE`	10 / 40	Range of random monster damage
`KEY_SCORE`	20	Score for finding the key in the cave
`TREASURE_SCORE`	50	Score for finding the treasure at the river
`score, health`	0, 100	The player's score and health, updated during the game

5. Implementation

The complete program is a single file, adventure_game.py, that uses only the Python standard library (random and argparse). The most important parts are shown below.

5.1 Input validation (forest function)

The choice is read as text and compared with the strings "1", "2" and "3". This avoids converting the input with int(), which would crash on text such as abc. Any other input, including an empty line, prints the error message and the loop asks again.

```
def forest():
    """Show the Forest menu and return a validated choice (1, 2 or 3).

    The player is asked again until a valid option is entered, so invalid
    input never affects health or score. If the input stream ends
    (Ctrl+D / Ctrl+Z), the game treats it as Quit.
    """
    print("\nYou are in the Forest. Where do you want to go?")
    print("1. Cave")
    print("2. River")
    print("3. Quit")

    while True:
        try:
            choice = input("Choice: ").strip()
        except EOFError:
            return QUIT

        if choice in ("1", "2", "3"):
            return int(choice)
        print("Invalid choice. Enter 1, 2 or 3.")
```

5.2 Cave and River

In the cave, health is reduced with max(0, health - damage) so it can never become negative. The key is awarded only if health is still above 0. The river always gives the treasure.

```
def cave(score, health, fixed_damage=None):
    """Cave: a monster attacks; if the player survives, a key gives +20.

    Returns the updated (score, health).
    """
    damage = get_damage(fixed_damage)
    health = max(0, health - damage)          # health never goes below 0
    print(f"Monster attacks! Damage: {damage}. Health: {health}")

    if health > 0:                            # survived the attack
        score += KEY_SCORE
        print(f"You found a key! Score: {score}")
    return score, health



def river(score, health):
    """River: a treasure gives +50 score.

    Returns the updated (score, health).
    """
    score += TREASURE_SCORE
    print("You found a treasure!")
    show_status(score, health)
    return score, health
```

5.3 Main game loop

The while loop keeps the game running while the player is alive. The if / elif / else chain sends the player to the correct place. The final score is displayed when the player quits and when the game is over.

```
def play(health=DEFAULT_HEALTH, fixed_damage=None):
    """Run one full game and return the final (score, health)."""
    score = 0

    print("=" * 40)
    print("       TEXT-BASED ADVENTURE GAME")
    print("=" * 40)
    print(f"You start in the Forest. Health: {health} | Score: {score}")

    while health > 0:                         # runs until quit or Health == 0
        choice = forest()

        if choice == CAVE:
            score, health = cave(score, health, fixed_damage)
        elif choice == RIVER:
            score, health = river(score, health)
        else:                                 # QUIT
            print(f"You left the adventure. Final score: {score} | Health: {health}")
            return score, health

    print(f"Game Over! Final score: {score}")
    return score, health
```

5.4 Random damage and testing mode

Random damage is produced by random.randint(10, 40). To make the test cases repeatable, the program accepts two optional command-line options:

```
python adventure_game.py                    # normal game
python adventure_game.py --health 30 --damage 30   # Test Case 2
python adventure_game.py --damage 30               # Test Case 4
```

6. Testing

The program was tested in two ways: by running the game manually from the console and with an automated test file (tests/test_adventure_game.py, written with Python's unittest module). All 18 automated tests pass.

6.1 Test cases from the problem statement

No.	Input	Expected output	Actual output	Result
1 Normal	Choice: 2 (River)	You found a treasure! Score: 50 | Health: 100	Same as expected	Pass
2 Boundary	Starting Health 30; Choice 1 (Cave); damage fixed to 30	Monster attacks! Damage: 30. Health: 0 Game Over! Final score: 0	Same as expected	Pass
3 Invalid	(a) Choice: 7 (b) Choice: abc	Invalid choice. Enter 1, 2 or 3. (for each) No health lost	Same as expected	Pass
4 Valid	Damage fixed to 30; Choices 1, 2, 3	Health 70; key, Score 20; treasure, Score 70; final score 70, Health 70	Same as expected	Pass

6.2 Additional edge-case tests

Scenario	Result
Monster damage over 1000 random calls	Always between 10 and 40
Damage larger than remaining health	Health stops at 0, never negative
Health 31 and damage 30 (survive with 1 health)	Key is still found; Score 20
Player dies in the cave	No key; score earned earlier is kept; loop stops without asking for more input
Inputs: empty line, space, 0, 4, -1, 1.5	All rejected with the error message; health unchanged
Input with spaces around a valid choice (" 2 ")	Accepted as choice 2
End of input (Ctrl+D) and Ctrl+C	Game exits cleanly without an error trace
200 randomised full games	Health always within 0 to 100; score always a multiple of 10

6.3 Console output of the test cases

Test Case 1: Normal input

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 50 | Health: 100

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 3
You left the adventure. Final score: 50 | Health: 100
```

Test Case 2: Boundary input (--health 30 --damage 30)

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 30 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 30. Health: 0
Game Over! Final score: 0
```

Test Case 3: Invalid input

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 7
Invalid choice. Enter 1, 2 or 3.
Choice: abc
Invalid choice. Enter 1, 2 or 3.
Choice: 3
You left the adventure. Final score: 0 | Health: 100
```

Test Case 4: Different valid scenario (--damage 30)

```
======================================
       TEXT-BASED ADVENTURE GAME
======================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 30. Health: 70
You found a key! Score: 20

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 70 | Health: 70

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 3
You left the adventure. Final score: 70 | Health: 70
```

7. Sample Runs

Two complete games with random monster damage are shown below. In the first the player quits; in the second the player keeps entering the cave until all health is lost.

Run A: player quits

```
========================================
       TEXT-BASED ADVENTURE GAME
========================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 20. Health: 80
You found a key! Score: 20

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 70 | Health: 80

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 40. Health: 40
You found a key! Score: 90

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 14. Health: 26
You found a key! Score: 110

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 3
You left the adventure. Final score: 110 | Health: 26
```

Run B: Game Over

```
=====================================
       TEXT-BASED ADVENTURE GAME
=====================================
You start in the Forest. Health: 100 | Score: 0

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 2
You found a treasure!
Score: 50 | Health: 100

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 17. Health: 83
You found a key! Score: 70

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 28. Health: 55
You found a key! Score: 90

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 27. Health: 28
You found a key! Score: 110

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 14. Health: 14
You found a key! Score: 130

You are in the Forest. Where do you want to go?
1. Cave
2. River
3. Quit
Choice: 1
Monster attacks! Damage: 21. Health: 0
Game Over! Final score: 130
```

8. Challenges and Solutions

Challenge	Solution
Wrong input such as abc would crash the program if converted with int()	Read the input as a string and compare it with "1", "2" and "3" inside a loop.
Random damage makes the test cases impossible to repeat	Added the optional --damage and --health command-line options so that damage and starting health can be fixed.
Health could become negative after a strong attack	Used max(0, health - damage) so that health stops at 0.

Challenge	Solution
The key must be given only if the player survives	Checked health > 0 after the attack before adding the score.
Checking the output by eye is slow and error-prone	Wrote automated unittest tests that feed scripted input and check the printed output.

9. Future Enhancements

- High-score table saved to a file.
- More places, items and an inventory, for example a healing potion.
- Difficulty levels that change monster damage.
- Coloured console output and, later, a graphical interface.

10. Conclusion

The Text-Based Adventure Game meets all the requirements of PS No. 39. It uses functions for each place, conditions for the player's choices, a loop that runs until the player quits or health reaches zero, and the random module for monster damage. Every menu choice is validated, and the final score is always displayed. The program passes all four given test cases and 14 further tests. The project helped me practise breaking a problem into functions, handling invalid input and testing code systematically, and it prepared the base for larger Python projects.

11. References

1. Python Software Foundation. random: Generate pseudo-random numbers. https://docs.python.org/3/library/random.html
2. Python Software Foundation. argparse: Parser for command-line options. https://docs.python.org/3/library/argparse.html
3. Python Software Foundation. unittest: Unit testing framework. https://docs.python.org/3/library/unittest.html
4. Python Software Foundation. The Python Tutorial. https://docs.python.org/3/tutorial/
5. Graphviz. Graph visualization software, used for the flowchart. https://graphviz.org